

Customer Problem Statement

CloudOps Sentinel

Enterprise Incident Response

Self-RAG Copilot

Problem Statement, Proposed Solution, Product Requirements, Technology Stack, and Deployment Scope

Customer	Environment	Primary Users	Use Case
Northstar Commerce Cloud (fictional)	24/7 cloud-native commerce platform	SRE, DevOps & Platform Engineering	Incident Response & Operational Support

Deployment target
Deploy the complete application on DigitalOcean Cloud using Docker, including FastAPI, the CloudOps Sentinel web UI, LangGraph Self-RAG workflow, SQLite persistence, Pinecone integration, OpenAI services, Tavily internet search, environment configuration, and production runtime.

1. Problem Statement

Northstar Commerce Cloud operates multiple production services supporting checkout, payments, catalog, identity, and order processing. Its SRE, DevOps, and platform teams maintain operational runbooks, standard operating procedures, deployment rollback guides, troubleshooting notes, and incident documentation. Although this knowledge already exists, engineers still spend valuable incident-response time searching for the correct document and deciding whether the retrieved guidance actually applies to the current failure.

Traditional keyword or semantic search can return multiple potentially related documents without determining whether the evidence is sufficient. A normal RAG chatbot introduces another risk: it may generate a confident troubleshooting recommendation from weak, incomplete, or unrelated retrieved chunks. During a production incident, such unsupported recommendations can increase investigation time and operational risk.

Some questions also require current public technical information that is not present in the private runbook knowledge base. For example, an engineer may need recent Kubernetes troubleshooting guidance or current platform behavior. The system therefore needs a controlled external-information path without replacing approved internal operational knowledge as the primary source.

Core business challenge
Engineers need fast and actionable incident guidance, while Northstar needs recommendations to remain grounded, verifiable, transparent, private-first, and capable of using current external information only when internal evidence is insufficient.

2. Why Simple RAG Is Not Enough

Simple RAG limitation	Problem in this project
Question -> Retrieve -> Generate	Retrieved chunks may be weak or incomplete, but the model may still generate an operational answer.
Retrieve once	The system does not automatically rewrite and retry when retrieval quality is poor.
Private KB only	Current technical information cannot be used when the internal runbooks do not cover the issue.

Simple RAG limitation	Problem in this project
No support verification	There is no explicit IsSUP check that the answer is supported by the evidence.
No usefulness verification	There is no explicit IsUSE check that the response actually resolves the engineer's question.
No persistent memory	Follow-up questions lose the previous incident context when requests are stateless.

3. Proposed Solution

Build CloudOps Sentinel - Enterprise Incident Response Self-RAG Copilot. The system will use LangGraph to execute a self-correcting retrieval workflow. It will restore the current incident session from SQLite, contextualize follow-up questions, retrieve trusted operational evidence from Pinecone, grade the retrieved evidence, rewrite weak queries, and use Tavily internet search only when the private knowledge base is insufficient.

After evidence is selected, the OpenAI language model generates the response. The workflow then performs IsSUP verification to check whether the answer is supported by the evidence and IsUSE verification to check whether the answer is useful for the user's question. Unsupported or unhelpful responses are revised/retried within a bounded retry policy rather than being returned blindly.

Step	Self-RAG behavior
1	Load persistent incident/conversation state from SQLite using the current thread ID.
2	Contextualize the user's question using prior incident context when required.
3	Retrieve relevant operational chunks from the private Pinecone knowledge base.
4	Grade whether the private evidence is relevant and sufficient.
5A	GOOD -> Generate an answer from private operational knowledge.
5B	WEAK -> Rewrite/refine the query and use Tavily internet search.
6	Grade the web evidence before using it.
7	Generate a grounded answer using the selected evidence.
8	Run IsSUP; revise/retry if the response is not sufficiently supported.
9	Run IsUSE; refine/retry if the response does not adequately answer the question.
10	Return the final answer, sources, workflow trace, and persist the updated session state.

4. Example - Answer Found in the Private Knowledge Base

Engineer: "Our checkout API started returning 502 errors after a production deployment. What should the on-call engineer check first?"

The Checkout API 502 Runbook exists in the private knowledge base. Pinecone retrieves the relevant chunks, the evidence grader accepts them, the OpenAI model generates a grounded response, and IsSUP/IsUSE validate the result. Tavily is not invoked because the private evidence is sufficient.

5. Example - Persistent Incident Memory

Follow-up: "What should I check next if that does not solve the issue?"

The application reuses the same thread ID. LangGraph restores the checkpoint from SQLite and uses the earlier checkout incident to contextualize the follow-up before continuing through the Self-RAG workflow.

6. Example - Private Knowledge Is Not Enough

Engineer: "What are the latest recommended troubleshooting steps for Kubernetes ImagePullBackOff errors?"

If the private runbooks do not provide sufficient evidence, the workflow rejects the weak context, rewrites the query, searches the internet through Tavily, grades the external evidence, generates a response, and still applies IsSUP and IsUSE before returning the answer.

7. Proposed Product Components

Technology / component	Role in CloudOps Sentinel
LangGraph	Self-RAG workflow orchestration, conditional routing, retry logic, graph state, and checkpoint integration.
OpenAI GPT-5 mini	Question contextualization, evidence grading, query rewriting, grounded generation, IsSUP, and IsUSE.
OpenAI text-embedding-3-large	Embeddings for operational documents and retrieval queries.
Pinecone	Private vector knowledge base containing CloudOps runbooks, SOPs, and operational guidance.
Tavily	Controlled internet-search fallback when private evidence is insufficient.
SQLite + LangGraph checkpointer	Demo persistence layer for thread-based conversation and workflow memory.
FastAPI	Backend APIs for chat, sessions, document upload, knowledge-base updates, and health/status.
HTML / CSS / JavaScript	CloudOps Sentinel incident-command-center web interface.
Document ingestion	PDF, TXT, Markdown, and DOCX loading, chunking, OpenAI embedding, and Pinecone upsert.
Docker	Packages the complete application and runtime consistently.
DigitalOcean Cloud	Hosts the containerized application.

8. Knowledge Base & Data Ingestion

The initial private knowledge base will contain representative operational documents including the Checkout API 502 Runbook, Payments High CPU Runbook, and Production Deployment Rollback SOP. A single `data_ingestion.py` script will load the documents, split them into chunks, generate OpenAI embeddings, and store the vectors in the configured Pinecone index and namespace.

The web UI will also allow new PDF, TXT, Markdown, and DOCX files to be uploaded. Uploaded documents must use the same ingestion and vector-store configuration and be added to the existing Pinecone knowledge base so they become available for retrieval without rebuilding the application.

9. Persistence Memory

LangGraph persistence will use SQLite for the demonstration. Each conversation will have a stable `thread_id`. The graph will compile with the SQLite checkpointer so conversation and workflow state can survive across API requests. The implementation should keep persistence replaceable so a production database can be introduced later.

10. User Interface Requirements

The CloudOps Sentinel UI should look like an incident-response product rather than a generic chatbot. It should provide incident-focused chat, example prompts, visible source routing, Self-RAG workflow/trace information, SQLite memory/session status, a New Session action, and a document-upload panel for extending the existing Pinecone knowledge base.

11. Expected Business Value

- Reduce time spent searching runbooks during production incidents.
- Help SRE and DevOps engineers reach evidence-backed next actions faster.
- Keep approved private operational knowledge as the primary source of truth.
- Reduce unsupported AI recommendations through evidence grading and IsSUP.
- Improve answer relevance through IsUSE and bounded self-correction.
- Handle private-knowledge gaps with controlled Tavily internet search.
- Preserve multi-turn incident context through LangGraph SQLite persistence.
- Allow operational teams to extend the knowledge base through UI document uploads.
- Provide sources and workflow traces for transparency and debugging.
- Deliver a production-style application rather than a notebook-only Self-RAG demo.

12. Acceptance Criteria

Acceptance test	Expected result
Initial ingestion	Running data_ingestion.py prepares the configured Pinecone knowledge base from the sample operational documents.
Private KB test	A checkout 502 question retrieves the internal runbook and does not invoke Tavily when the evidence is sufficient.
Memory test	A follow-up question in the same thread uses the previous incident context restored from SQLite.
Internet fallback test	A question not sufficiently covered by the private KB can trigger query rewriting and Tavily search.
Self-RAG validation	Generated answers are evaluated by IsSUP and IsUSE and revised/retried when necessary.
Dynamic KB update	A supported document uploaded from the UI is added to the same Pinecone knowledge base and becomes retrievable.
Transparency	The application returns source information and workflow/route trace with the final response.
Deployment	The complete FastAPI + UI application can be Dockerized and deployed to DigitalOcean.

13. Deployment - DigitalOcean + Docker

The final application will be containerized with Docker and deployed to DigitalOcean Cloud. Production configuration will provide OpenAI, Pinecone, Tavily, and application settings through environment variables. The deployed system will demonstrate the complete lifecycle: initial ingestion, private retrieval, Self-RAG grading and correction, persistent memory, controlled web fallback, dynamic document upload, answer traceability, and application logging.

Deployment step	Activity
1	Build and test the complete application locally.
2	Run data_ingestion.py and validate the Pinecone knowledge base.
3	Create the Dockerfile/container image.
4	Configure required environment variables and API credentials.
5	Provision the DigitalOcean runtime.
6	Deploy and run the Dockerized FastAPI + web UI application.
7	Persist the SQLite checkpoint database for the demonstration.
8	Verify retrieval, memory, Tavily fallback, Self-RAG validation, document upload, sources, and workflow trace.

14. FDE Perspective

The objective is not simply to implement Self-RAG in a notebook. The Forward Deployed Engineer starts from the customer's incident-response problem and delivers a usable product: operational knowledge ingestion, OpenAI services, Pinecone retrieval, Self-RAG orchestration, persistent memory, controlled internet search, APIs, a domain-specific UI, testing, Docker packaging, DigitalOcean deployment, observation, and continuous improvement.

End-to-end delivery path

Customer Incident Problem -> Understand Workflow -> Connect Runbooks -> Build Self-RAG -> Add SQLite Persistence -> Add Controlled Internet Search -> Expose APIs -> Build CloudOps UI -> Test -> Dockerize -> Deploy on DigitalOcean -> Observe -> Improve